Gm.

Sui & Move Bootcamp

Two days. Build. Ship on-chain.

Two-Day Overview

Day 1 - Sui and Move

Publishing your first move package

- Module 1: Sui, Move, and objects
- Module 2: Move Fundamentals & Syntax
- Module 3: Abilities & Patterns
- Module 4: Test Modules
- Module 5: Deployment & CLI
- Hackathon Reveal + Homework

Day 2 — The Builder

Bridging Move to the World

- Module 1: Sui TypeScript SDK
- Module 2: Sui TS gRPC Read + Write Queries
- Module 3: Bootstrap a Sui dApp Repo (Sui dApp Vite + React Template)

Day 1

The Sui Architect

Thinking in Objects, not Accounts

Sui, Move, and objects

Module 1

Sui & Move in 60 Seconds

Sui

- **Layer 1** — purpose-built for **fast, parallel** execution and **low-latency** apps
- **Object-centric** — state scales without one global account bottleneck
- **Mysten Labs** — created Sui; leads core protocol development

Move

- **Smart-contract language** — Rust-like syntax; **resource-safe** by design
- **Origin: Meta** (Diem); assets as **typed values**, not opaque storage
- **Sui Move** — Mysten's **dialect** for Sui's **object model**

Sui Ecosystem

Part 1 of 2 — storage, privacy, identity, onboarding.

Walrus

Decentralized **blob storage** network — cheap, durable large files with **onchain attestations**, built for data-heavy apps without stuffing bytes into objects.

Seal

Encryption & access control for onchain data — **policy-based decryption** so apps can share secrets without exposing plaintext to the chain.

SuiNS (Sui Name Service)

Human-readable `.sui` names mapped to addresses and identity — **one name** instead of hex strings for wallets, apps, and branding.

Enoki

Embedded wallets + zkLogin + gas sponsorship in one product surface — **Web2-style onboarding** with **Sui-native** security and fewer “install a wallet first” dead ends.

Sui Ecosystem

Part 2 of 2 — markets, compute, wallet.

DeepBook

Native central limit order book (CLOB) on Sui — **order-book liquidity** and price discovery **without** porting Ethereum AMM mental models wholesale.

Nautilus

Verifiable off-chain compute tied to Sui — run heavy or sensitive logic in **TEEs** (or similar) and **prove correctness on-chain** for apps that can't fit everything in Move gas limits.

Slush Wallet

Multi-chain wallet experience with strong **Sui-first** design — **one place** to connect to Sui dApps while still playing nicely with broader crypto habits.

What makes Sui different?

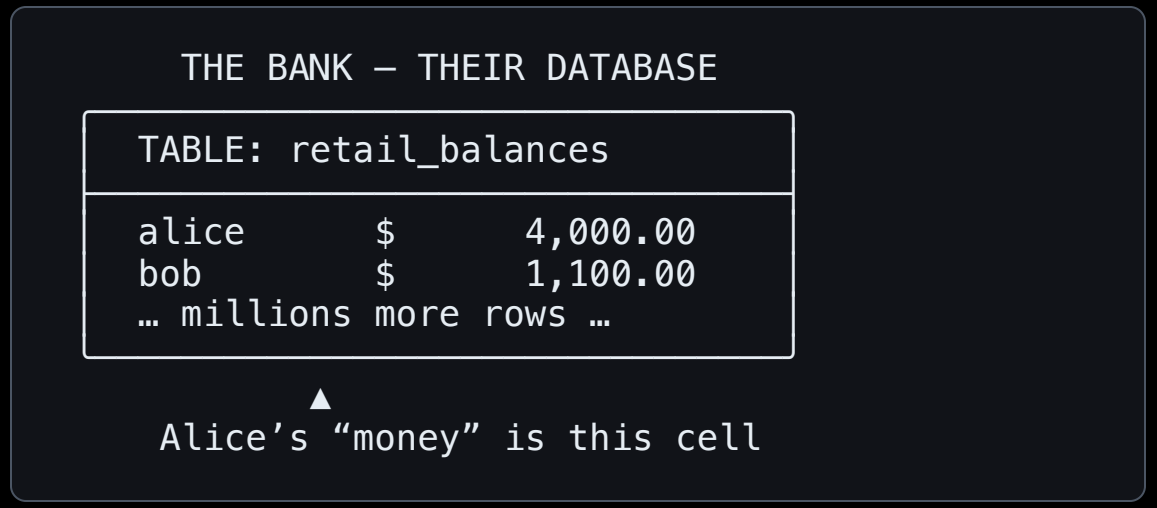
- **Object-centric model (not account-centric):** assets are real on-chain objects with explicit ownership, not just rows in shared contract storage.
- **Parallel by design:** transactions touching different objects can execute in parallel, improving throughput without forcing every app into one global bottleneck.
- **Fast finality + low-latency UX:** many common transactions settle quickly, making app interactions feel closer to Web2 expectations.
- **Safer smart contracts with Move:** resource-oriented types and strict ownership rules reduce common classes of contract bugs.
- **Better onboarding primitives:** sponsored transactions and zkLogin-style flows make mainstream user experiences more practical.

Accounts vs Objects - an analogy

From bank balances to physical items

Bank balance - Account

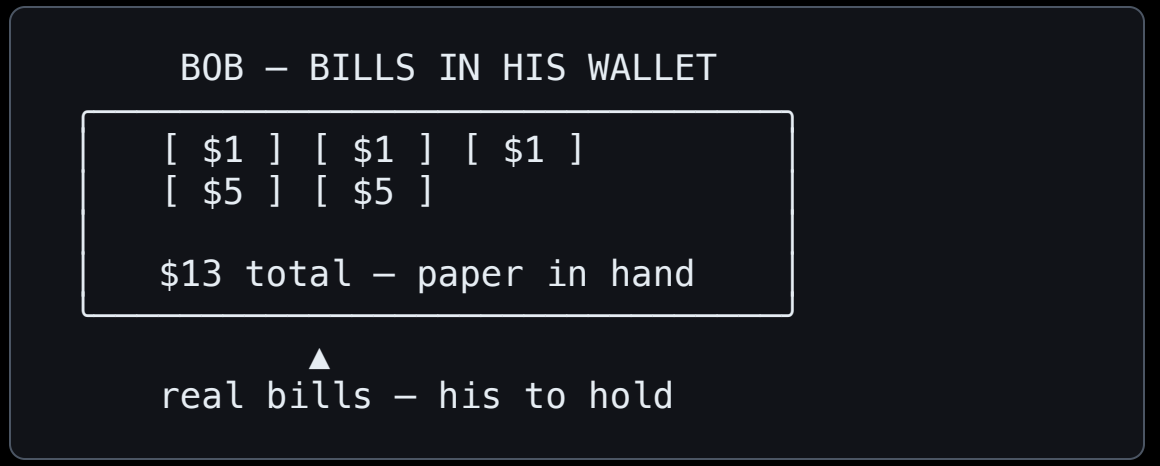
Alice — she only ever sees an app. The “money” lives elsewhere.



> You hold a story about a row in their spreadsheet.

Physical Items - Sui

Bob — **cash in his wallet**: ones and fives he can **spread out and count** — not a line in anyone’s ledger.



> You hold the money — not a story about a balance.

Account Model vs Object Model

Why Sui is different from other blockchains?

Ethereum-style (account-centric)

- state is attached to accounts and contract storage slots
- balances and mappings are looked up from shared global state
- common pattern: "read account/storage, update value, write back"

Sui (object-centric)

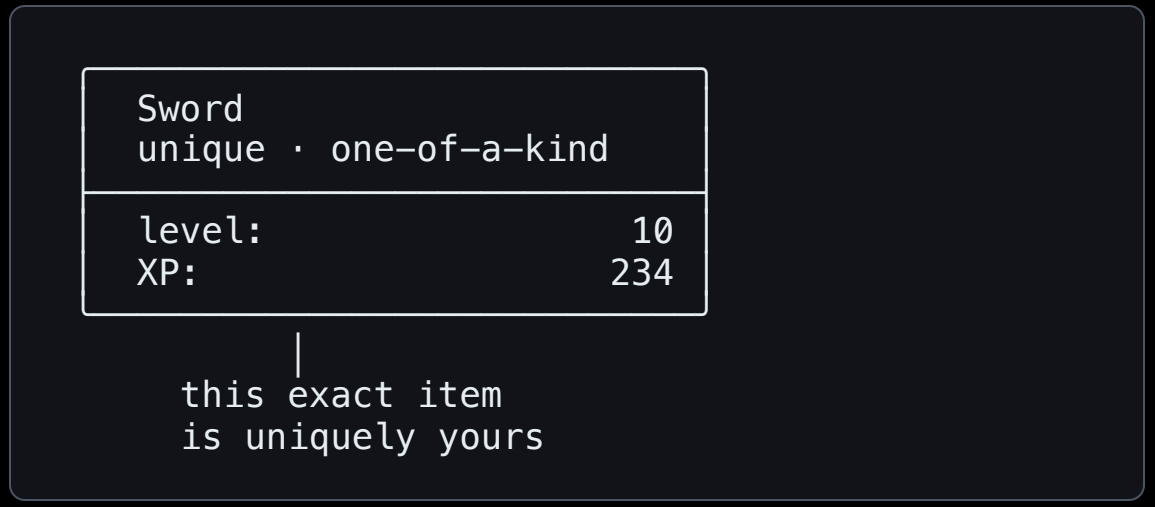
- assets are first-class on-chain objects with IDs
- ownership is explicit: address-owned, shared, or immutable
- transactions directly use object references as inputs/outputs

What is an object

Everything is a 'thing'

- **Your fungible tokens (USDC)**

- **Your NFTs**



Why this matters

Security through ownership

Smart contract hacking

- **Other chains:** A hacker exploits the contract and drains the balance — **your balance is now 0**.
- **Sui:** Your object **can't be moved** due to the ownership checks. **Only you** can modify your objects. This is **enforced by the source code**.

Objects Are Composable

You can **wrap objects inside objects** to model nested rights and workflows.

```
public struct Badge has key { id: UID, level: u8 }

public struct Backpack has key {
    id: UID,
    badge: Badge, // object composition
}
```

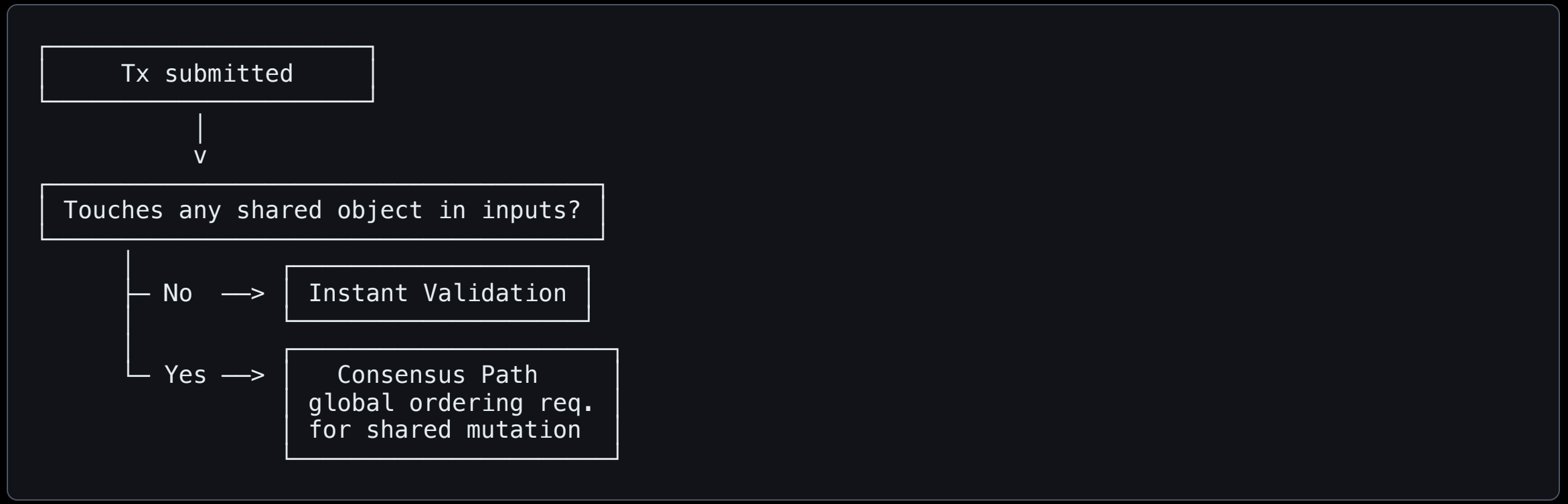
In account-based models, composition is usually indirect (mappings/IDs).
In Sui, composition is native to the type system.

Object States

- **Owned**: in a wallet/another object (wrapped/owned by a wallet).
- **Shared**: many users can access; ordering/consensus is required.
- **Immutable**: readable by all, mutable by none.

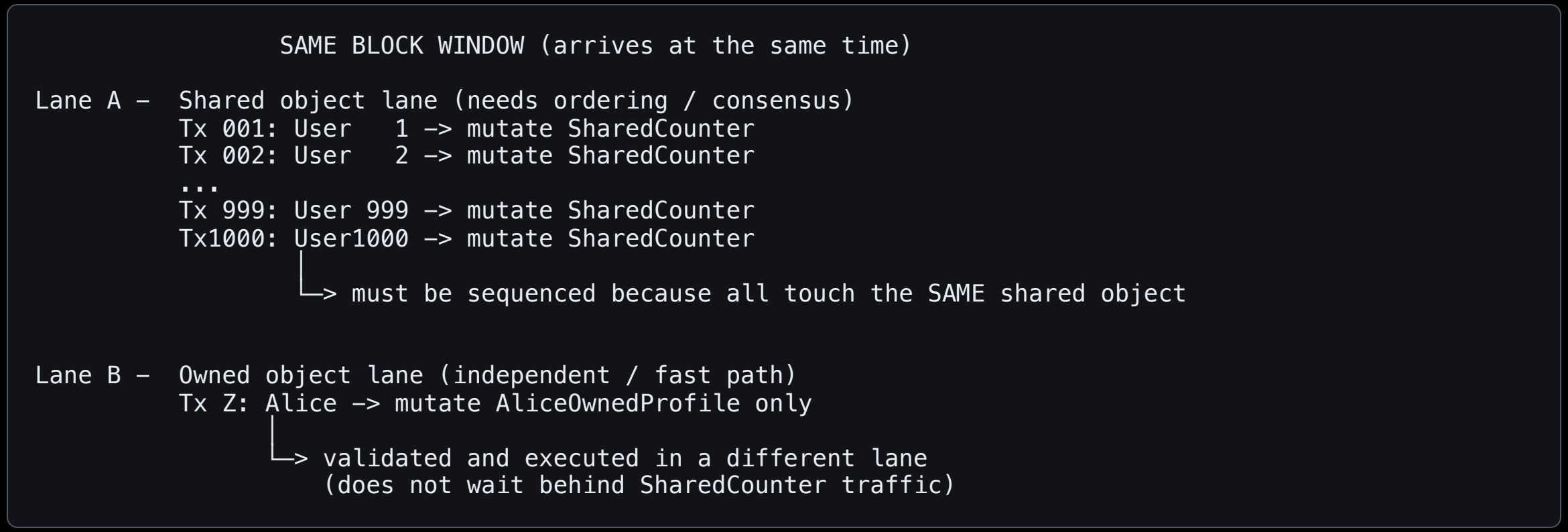
Why this matters: state type determines performance profile, access model, and API design.

Mysticeti Consensus Protocol (And Parallelism)



Rule: shared object touched → consensus path; no shared object touched → Instant validation.

Parallelism in Sui



- 1000 users contending on one shared object create a serialization bottleneck in that shared lane.
- A transaction that only touches owned objects can still proceed on its own lane in parallel.
- Design implication: keep user-specific state owned; share only state that truly needs coordination.

Which needs consensus?

```
Independent owned objects
  Tx A: mutate Object X (Alice)
  Tx B: mutate Object Y (Bob)

Same shared object
  Tx A: mutate Shared Pool P
  Tx B: mutate Shared Pool P
```

Design rule: keep state owned by default; introduce shared objects only for real coordination.

Module 1 Quiz — Scenario 1

You are building a peer-to-peer item marketplace.

A teammate proposes:
"Let's store all listings in one shared global object so querying is easy."

Another proposes:
"Each listing should be an owned object, and we only share what must be shared."

Decision: Which design do you choose first, and why?

Module 1 Quiz — Scenario 2

Your app has two features:

- Personal loadout customization (user-specific)
- Global tournament leaderboard (everyone updates)

Decision: Which state should remain owned objects, which should be shared, and why?

Think:

- when coordination is truly required
- when over-sharing introduces unnecessary consensus overhead

From Sui Concepts To Move Code

Module 2 — Move Fundamentals & Syntax

Move Package Anatomy & Compiler Flow

Scaffold and start:

```
sui move new my_first_package
cd my_first_package
sui move build
sui move test
```

```
my_first_package/
  Move.toml          # package manifest
  sources/
    hero.move        # modules
  tests/
    hero_tests.move  # optional
```

Compiler view: `Move.toml -> modules -> bytecode -> publish to package address`.

Packages, Modules, and Members

- A published package has an on-chain address (for example: `0xabc...`).
- Modules live inside that package: `0xabc...::hero`.
- Functions/types are addressed from that module:
  - `0xabc...::hero::mint`
  - `0xabc...::hero::Hero`

Mental model: **package::module::member**.

Data Types

Primitive types

- unsigned integers: `u8`, `u16`, `u32`, `u64`, `u128`, `u256`
- `bool` for true/false conditions
- `address` for on-chain account/object addresses

Non-primitive types

- `vector<T>` for dynamic collections
- `String` (from stdlib) for text values
- `struct` for custom domain models (`Hero`, `Badge`, etc.)

Data Types (Example)

Pick the smallest type that matches business reality:

```
const MAX_LEVEL: u8 = 100;

public struct Hero has key, store {
    id: UID,
    level: u8,          // small bounded domain
    xp: u64,            // grows over time
    achievements: vector<Badge>
}
```

Overflow: What It Is and How To Prevent It

In Move, integer overflow/underflow aborts execution.

```
public fun bad_add(a: u8, b: u8): u8 {
    // aborts if a + b > 255
    a + b
}
```

Prevent with wider accumulator types:

```
public fun add_points(level: u8, delta: u8): u8 {
    let next = (level as u64) + (delta as u64);
    assert!(next <= 100, EOverflow); // business bound first
    next as u8
}
```

Decision rule: choose type + bounds together, then enforce with `assert!`.

Why `u8` Maps to 256 Values

```
u8 = unsigned 8-bit integer

Binary min: 00000000₂ = 0
Binary max: 11111111₂ = 255

8 bits  ->  2^8 combinations  ->  256 total values
```

Key idea:

- `256` = number of possible values
- `255` = largest value representable in `u8`

Strings

- Prefer method syntax from Move 2024: `b"Hero".to_string()`.
- If you only use `b"Hero"` it is actually a `vector<u8>` and not a `String`.
- `String` is not auto-imported; `use std::string::String`.

Struct

```
public struct Hero has key, store {
    id: UID,
    name: String,
    level: u8,
}
```

```
public fun mint_hero(name: String, ctx: &mut TxContext) {
    Hero {
        id: object::new(ctx), // runtime generates unique object ID
        name,
        level: 1,
    }
    transfer::public_transfer(hero, tx_context::sender(ctx));
}
```

- `struct` only defines the data model/type.
- `object::new(ctx)` is called at creation time to produce a fresh `UID`.
- `ctx` (`&mut TxContext`) is injected by the runtime per transaction.

Unpacking a Struct

Unpacking destructures fields out of a value **by consuming it**.

```
public fun burn_hero(hero: Hero) {
    let Hero { id, name: _ } = hero;
    object::delete(id);
}
```

After unpacking, `hero` no longer exists as a whole value (ownership moved to fields).

Real-life use case: **account closure / object burn**.
When a profile or membership is deactivated, unpack to access `id` and delete the object cleanly.

Control Flow Expressions

1. if else

```
if (level > 10) {
    bonus = 2;
}
```

```
let rank = if (xp >= 1000) "pro" else "rookie";
```

```
if (score >= 90) {
    grade = 1;
} else if (score >= 75) {
    grade = 2;
} else {
    grade = 3;
};
```

Control Flow Expressions (Loops + Guards)

2. while

```
let mut i = 0;
while (i < 3) {
    i = i + 1;
};
```

3. loop + break

```
let mut n = 0;
loop {
    if (n == 5) break;
    n = n + 1;
};
```

4. guard-style checks

```
assert!(amount > 0, EInvalidAmount);
if (!is_admin) return;
```

Use guard clauses early; keep nested branches shallow.

Function Kinds and Modifiers

Who can call what:

Signature	Internal (same module)	External Move (other modules/packages)	Transaction (PTB/CLI/SDK)
`fun f()`	Yes	No	No
`public fun f()`	Yes	Yes	Yes
`public(package) fun f()`	Yes	Same package only	No
`entry fun f()`	Yes	No	Yes

When to use:

1. `public fun`: reusable business logic/API

```
// Strong use case: keep policy math reusable by other modules and tests.
public fun compute_training_cost(level: u64): u64 {
    10 + level * 2
}
```

- Use this when you want composability from other Move modules.
- Good for shared logic, validation helpers, and read-only or deterministic rules.

2. `entry fun`: user-facing transaction route

```
// Strong use case: state-changing user action from wallet/PTB.
entry fun train(hero: &mut Hero, points: u64, ctx: &mut TxContext) {
    let sender = tx_context::sender(ctx);
    assert!(sender == hero.owner, 0);
    hero.level = hero.level + points;
}
```

- Use this when the function is a direct on-chain action users execute from wallet/SDK/CLI.
- Use non-`public` `entry` to prevent other packages from wrapping your call in their Move logic.

Function Calls, References, and Ownership

Before Function Runs: What Gets Checked

```
public fun add_cert_into_my_passport(
    cert: Certificate,
    obtained_at: u64,
    passport: &mut Passport,
    ctx: &mut TxContext
) {
  // if this function runs, all checks already passed.
}
```

When a PTB calls your Move function, validators + VM checks include:

- function exists
- argument count and types matches
- object inputs actually exist
- **sender/ownership permissions match object kind (owned/shared/immutable)**

These checks happen before entering the function body.
If any check fails, the transaction is rejected and zero lines inside the function execute.

Passing an Object into a function

- Pass by `&` / `&mut` when you only need to read or mutate in-place.
- Pass by value when ownership should move.
- We include objects as params so function access is explicit and verifiable.

```
// GOOD: borrow mutably, caller keeps ownership
fun sharpen_sword(sword: &mut Sword, amount: u64) {
    sword.power = sword.power + amount;
}

// BAD: by-value param when mutation-in-place is intended
fun sharpen_sword_bad(sword: Sword, amount: u64) {
    let mut s = sword;
    s.power = s.power + amount;
}
```

By-value params (`sword: Sword`) consume ownership.
Reference params (`sword: &Sword` / `&mut Sword`) do not.

Returning Values in Functions

```
// 1) tail expression return (implicit)
public fun add(a: u64, b: u64): u64 {
    a + b
}

// 2) explicit return
public fun add_explicit(a: u64, b: u64): u64 {
    return a + b;
}

// 3) early return guard
public fun bonus(is_admin: bool): u64 {
    if (!is_admin) return 0;
    100
}
```

Return Styles You Will Use Often

```
// 4) no return value (unit: ())
public fun level_up(hero: &mut Hero) {
    hero.level = hero.level + 1;
}

// 5) return multiple values
public fun split_points(total: u64): (u64, u64) {
    (total / 2, total - (total / 2))
}

// 6) return resource ownership
public fun rename(hero: Hero, name: String): Hero {
    let mut h = hero;
    h.name = name;
    h
}
```

Use returns to make ownership flow explicit and predictable.

Import Syntax

```
use my_pkg::hero;                          // module
use my_pkg::hero::Hero;                    // one member
use my_pkg::hero::{Self, Hero, ENotAdmin}; // module + many members
use sui::test_scenario as ts;              // alias (rename)
use std::option::{Self, Option as Maybe};  // alias specific member
```

- Path shape is always `package::module::member`.
- `Self` imports the module namespace itself (for `hero::mint()` style calls).
- Braces import multiple members in one line.
- `as` is only for renaming; use it when names are long/conflicting.

(For Import Examples)

```
module my_pkg::hero;

public struct Hero has key, store { id: UID, level: u64 }
#[error] const ENotAdmin: u64 = 0;

public fun mint(ctx: &mut TxContext): Hero { /* ... */ }
public fun level_up(hero: &mut Hero) { /* ... */ }
public fun assert_admin(sender: address) { /* ... */ }
```

Next slides import from this same module using different syntax.

Import Example 1: Module Namespace (`Self`)

```
use my_pkg::hero::{Self}; // imports module namespace `hero`
use my_pkg::hero; // imports module namespace `hero`

public fun train(ctx: &mut TxContext) {
    let mut h = hero::mint(ctx); // call via module namespace
    hero::level_up(&mut h);
}
```

`Self` means "import the module itself", so `hero::...` works.

What's imported

```
module my_pkg::hero;
// members...
```

Import Example 2: Single Member

```
use my_pkg::hero::Hero; // imports one member only

public fun read_level(h: &Hero): u64 {
    h.level
}
```

Import just what you need when only one type/function is used.

What's imported

```
public struct Hero has key, store {
    id: UID, level: u64
}
```

Import Example 3: Module + Members Together

```
use my_pkg::hero::{Self, Hero, ENotAdmin};

public fun guarded_train(sender: address, h: &mut Hero) {
    hero::assert_admin(sender);
    assert!(h.level < 100, ENotAdmin);
    hero::level_up(h);
}
```

Braces let you pull module namespace and selected members in one line.

What's imported

```
module my_pkg::hero;
// namespace (`Self` → hero::…)
```

```
public struct Hero has key, store
{ ... }
```

```
#[error]
const ENotAdmin: u64 = 0;  // `ENotAdmin`
```

Import Example 4: `as` Alias (Rename)

```
use sui::test_scenario as ts;
use std::option::{Self, Option as Maybe};

public fun demo_alias() {
    let _s = ts::begin(@0xA);           // short alias for long module path
    let _m: Maybe<u64> = option::none(); // renamed type
}
```

Use `as` only when renaming improves readability or avoids name conflicts.

Dot Notation Function Calls

```
public fun train(h: &mut Hero) {
    hero::level_up(&mut h); //Traditional way
    h.level_up(); //Easier way
}
```

Rules:

```
//1.
public fun level_up(hero: &mut Hero,...) {
    ...
    hero.level = hero.level + 1;
    ...
}
//2. Hero is declared in the same module as the function you're calling
```

Move 2024 Auto Imports

These are implicitly available in each module:

```
use std::vector;
use std::option::{Self, Option};
use sui::object::{Self, ID, UID};
use sui::transfer;
use sui::tx_context::{Self, TxContext};
```

TxContext: What It Contains

`TxContext` is VM-provided metadata for the current transaction.

- `sender(ctx)` → signer address
- `digest(ctx)` → tx digest (`vector<u8>`, not randomness)
- `epoch(ctx)` → current epoch number
- `epoch_timestamp_ms(ctx)` → epoch start timestamp
- `sponsor(ctx)` → `Option<address>` if gas sponsor exists
- `fresh_object_address(ctx)` / `object::new(ctx)` → unique object IDs

It cannot be manually constructed in contracts; the runtime injects it.

Mutation Syntax: Field Access vs `*ref`

```
public fun train(hero: &mut Hero, xp_ref: &mut u64) {
    // style 1: mutate object fields directly
    hero.level = hero.level + 1;

    // style 2: mutate through a reference variable
    let final_xp = *xp_ref + 10;
}
```

- `hero.field = ...` when you have a struct/object reference and want a field update.
- `*ref = ...` when the variable itself is a reference to a primitive/value.
- For object members, prefer field syntax; for standalone refs, use `*`.

Important Rule: No Dangling Resources

If a function takes an object **by value**, it must be:

- returned, or
- transferred/stored, or
- explicitly destroyed (if allowed).

Only references (`&` / `&mut`) can end without explicit consume logic.

Dangling Resource Examples

```
// GOOD: moved object is transferred
public fun send_hero(hero: Hero, to: address) {
    transfer::public_transfer(hero, to); // send to addr
    transfer::transfer(hero, to); // send to addr
    transfer::share_object(hero); // convert into shared object
}

// BAD: moved object is never consumed
public fun hero_level_up(hero: Hero) {
    hero.level = hero.level + 1;
    // ERROR: hero is left hanging at function end
}
```

The compiler rejects `forget_bad` because `Hero` lacks `drop`.

Dangling Resource Examples

```
public fun mint(ctx: &mut TxContext) {
    let hero = Hero { id: object::new(ctx), name: b"A".to_string() };
    // ERROR or no error?
}

public fun burn(hero: Hero) {
    let Hero { id, name: _ } = hero;
    object::delete(id);
    // ERROR or no error?
}

public fun replace(mut hero: Hero, ctx: &mut TxContext) {
    hero = Hero { id: object::new(ctx), name: b"B".to_string() };
    // ERROR or no error?
}
```

Rule: every by-value resource must end as return, transfer/store, or explicit destruction.

Module 2 Quiz — Scenario 1

You are designing an object for proof-of-work records:

- volunteering effort
- hackathon participation
- event attendance badges

Decision: pick Move field types for:

- `volunteer_hours`
- `hackathons_joined`
- `event_badges`

Then justify each with safety reasoning:

- why your numeric choices reduce overflow risk
- why your badge representation fits growth and query needs

Module 2 Quiz — Scenario 2

You need `add_volunteer_proof(...)` to append a new verified proof into a user's existing `Passport`.

A teammate proposes:
"Pass `passport` by value so the function can do anything."

Another proposes:
"Pass `&mut Passport` because we mutate in place and preserve ownership."

Decision: which function parameter style is correct, and why?

Abilities & Patterns

Module 3 — Ownership Semantics

Ability

Abilities are not syntax decoration; they are security boundaries.

In Sui, the four you must reason about first are:

- `key`: object identity and object existence on-chain
- `store`: persistence and public storable/transferable behavior
- `copy`: whether duplication is allowed
- `drop`: whether silent discard is allowed

Ability: `key`

Defines an on-chain top-level object with identity (`UID`).

Correct:

- `Hero has key { id: UID, ... }`
- `ManagerCap has key { id: UID }`

Wrong:

- `Broken has key { count: u64 }` (missing `id: UID`)

Ability: `store`

`store` controls whether a value can be embedded in other persistent on-chain state and publicly transferred.

If a type lacks `store`, users cannot use `public_transfer`, and only the creator module can transfer it.

Ability: `store` — What Is Possible

- child can be wrapped by persistant parent object, ONLY if child has `store`
- `transfer::public_transfer` can only happen on objects that has `store`
- objects without store can ONLY be transferred by `transfer::transfer`

Ability: `store` — Scenarios

Scenario A (allowed):

- `Metadata has store { name: String }`
- `Profile has key, store { id: UID, meta: Metadata }`
- `meta` is valid in persistent object state because it is `store`

Scenario B (not allowed):

- `SoulboundID has key { id: UID }` (no `store`)
- `Wrapper has key, store { id: UID, soul: SoulboundID }`
- invalid because non-`store` is embedded into persistent state

Scenario C (allowed but module-controlled):

- `SoulboundID` has no `store`
- defining module calls `transfer::transfer(soulbound, owner_or_object)`
- module-internal movement/wrapping is possible, but no public transfer path

Ability: `copy`

Allows duplication by value.

- numeric primitives (`u64`) are copyable

```
// Example
public struct BadgeScore has copy, drop, store {
    points: u64,
}

public fun copy_is_legit(score: BadgeScore):{
    // struct has `copy`, so copying by assignment is legal
    let s1 = score;

    let p1 = s1.points;
    (s1, p1)
}
```

Ability: `drop`

Allows values to be silently discarded at end of scope.

```
public struct TempTag has drop {}
public struct Params has copy, drop, store { retries: u8 }

public fun legit_drop_cases() {
    // Case 1: explicit ignore
    let t = TempTag {};
    _ = t;

    // Case 2: dropped at end of scope (no use)
    let _unused = TempTag {};

    // Case 3: overwrite old value; previous value is dropped
    let mut p = Params { retries: 3 };
    p = Params { retries: 5 };
    _ = p;
}
```

Ability Patterns In Real Systems

- **OTW (One-Time Witness):** enforce one-time initialization patterns.
- **Capabilities:** make authority an object (`AdminCap`, `ManagerCap`).
- **Soulbound:** constrain transfer semantics via ability choices.
- **Hot Potato:** zero-ability receipts that must be consumed.

Abilities are your first security layer, not just syntax decoration.

What Is An OTW?

OTW (one-time witness) — a special pattern that proves: "this code is running in the module's **first** transaction (publish), not later." The runtime hands you **exactly one** value of that type, only in `init()`.

```
module demo::thing;

public struct THING has drop {}

use sui::package;

fun init(otw: THING, ctx: &mut TxContext) {
    package::claim_and_keep(otw, ctx);

    // same result below
    let publisher = package::claim(otw, ctx);
    transfer::public_transfer(publisher, ctx.sender());
}
```

OTW → `Publisher` Authority

The OTW is **consumed** when you turn it into durable **publisher** proof: a `sui::package::Publisher` object (`key` + `store`) that records **which package and module** published a type. Other code passes `&Publisher` into APIs that must only trust the real publisher (`from_module` / `from_package` checks).

Using The `Publisher` Object

Administrative powers (gated entrypoints) — any "publisher-only" action checks the caller holds `&Publisher` for **this** package/module before minting, upgrading policy, or changing module-owned registries.

```
const ENotPublisher: u64 = 0;

public fun mint_a_thing(pub: &Publisher, name: String, ctx: &mut TxContext): Thing {
    assert!(pub.from_module<THING>(), ENotPublisher); // same module as OTW
    Thing { id: object::new(ctx), name }
}
```

`from_package` / `from_module` tie the capability to **bytecode identity**, not to a secret string in your repo.

OTW - Rules

Rules (Sui Move):

- **Name:** the struct must match the **module's name** in **ALL_CAPS** (same spelling as the last segment of `module a::b::name` → `NAME`).
- **Abilities:** must have `drop` (and typically nothing else on the witness itself).
- **Where it appears:** only as the first argument to `init(otw: YOUR_OTW, ctx: &mut TxContext)` — you do not construct it by hand elsewhere; the system supplies it once at publish.
- **Framework check:** APIs that require a real OTW use `types::is_one_time_witness(&otw)` — arbitrary structs you mint yourself will **not** pass.

Empty witness is fine: `public struct MYMOD has drop {}` or `public struct MYMOD() has drop {}`.

The Capability Pattern

Digital authority is a physical asset you can hold, transfer, or burn.

- A `ManagerCap` is just an object in your wallet
- Functions gate themselves by requiring it as a parameter
- Lose the cap — lose the authority. Transfer it — transfer the power.

```
public fun admin_only(_cap: &ManagerCap, /* ... */) {
    // only callable if caller holds ManagerCap
}
```

Soulbound — What & Why

- **Name comes from games:** a soulbound item is **bound to your character** — you can't trade it, mail it, or flip it on an auction house; it **stays on that identity**.
- **Same idea in crypto: non-transferable** tokens — credentials, attestations, reputation — **not** liquid collectibles you freely sell.
- **On Sui / Move:** "soulbound" is **not** a keyword — you get the behavior from **abilities** on your struct.
- **No `store`:** users **cannot** call `public_transfer` on the object themselves; it won't behave like a normal tradable asset in the wallet.
- **Module stays in control:** only **your module** can move the object (e.g. `transfer::transfer`) through **entrypoints you write** — e.g. migration, recovery, or a deliberate handoff.

Soulbound — Example

Struct with `key` but **no** `store`: mint into the user's inventory, but **only the module** can relocate it.

```
public struct SoulboundID has key {
    id: UID, // no `store`
}

public fun mint(ctx: &mut TxContext): SoulboundID {
    SoulboundID { id: object::new(ctx) }
}

public fun module_transfer(id: SoulboundID, to: address) {
    transfer::transfer(id, to); // module-controlled path
}
```

`module_transfer` is **your** escape hatch — not something arbitrary callers can do without your logic.

Hot Potatoes Pattern

- A **hot potato** type is a struct with **no abilities** — not `copy`, `drop`, `store`, or `key`.
- **You can't ignore it:** can't drop it at end of scope, stash it in an object, or copy it — it must be **moved forward** until something **consumes** it legally.
- **Why the name:** like the **party game** — you **can't hold** the potato; you **pass it on** until someone **finishes** the round. In Move, the "round" is usually **one transaction**: the value **must** reach a function that **destroys or uses** it, or the tx **won't compile** (or will **abort** if a path forgets it).
- **What it's for: linear protocols** — borrow → act → **must** repay; upgrade **ticket** → **must** commit; rule checks that **can't** be skipped silently.

Hot Potatoes — Flash Loan (Abstract)

`FlashLoanReceipt` is the **hot potato**: it carries **how much** was borrowed (and **which pool**) so **payback** can't be "forgotten" in the happy path.

```
/// No abilities → cannot drop / store / copy; must flow to `repay_flash_loan`.
public struct FlashLoanReceipt {
    borrowed: u64,   // principal taken from the pool
    min_repay: u64,  // principal + protocol fee (what must come back)
    pool: ID,        // which liquidity pool owns the debt
}

/// Pull liquidity + mint the receipt in one step.
public fun borrow_flash(pool: &mut LiquidityPool, amount: u64, ctx: &mut TxContext) : (Coin<SUI>, FlashLoanReceipt){
    ...
    (coin, receipt)
}

/// Consumes receipt and returns funds — receipt disappears only here.
public fun repay_flash_loan(pool: &mut LiquidityPool,receipt: FlashLoanReceipt,payment: Coin<SUI>) {
    ...
    let FlashLoanReceipt { borrowed: _, min_repay: _, pool: _ } = receipt;
}
```

Callers **thread** `(Coin, FlashLoanReceipt)` through their logic; **until** `repay_flash_loan` runs, the receipt **has nowhere to go** except forward — that's the **enforced** borrow–repay story.

Module 3 Quiz — Scenario 1

You are designing `TreasuryAdminCap` for a DAO:

- Phase 1: founder-only admin authority
- Phase 2: authority can move to multisig after governance approval

Decision tasks:

1. Should this cap be publicly transferable by default?
2. Should movement be module-controlled?
3. What changes between Phase 1 and Phase 2?

Explain your answer with ability + transfer-path reasoning.

Module 3 Quiz — Scenario 2

Given:

```
public struct ManagerCap has key {
    id: UID,
    role: u8
}
```

Requirements:

1. External users must not be able to `public_transfer` this cap.
2. Only the module can decide when and to whom this cap moves.

Decision tasks:

1. Does the struture abilities satisfy the requirements? Why?
2. How would you design for requirement #2? hint: when you write the transfer function, do you take in the ManagerCap by reference or by value?

Test Modules

Module 4 — Build Confidence Before Publish

Test Module Structure

```
my_pkg/
  sources/hero.move
  tests/hero_tests.move
```

```
#[test_only]
module my_pkg::hero_tests;
use my_pkg::hero;
use sui::test_scenario as ts;
```

Use tests for behavior, permissions, and state transitions before deploying.

Test Mental Model (Like Unit Tests)

Think of a Move test exactly like classic unit testing:

- **Setup**: prepare addresses, start `test_scenario`, seed initial objects/state.
- **Act**: call the function(s) under test (mint, transfer, level_up, etc.).
- **Assert + Teardown**: read state, assert expectations, return objects, end scenario.

```
#[test]
fun test_mint_single_tx() {
  // SETUP

  // ACT

  // ASSERT + TEARDOWN
}
```

This lens helps you design tests from behavior first, not syntax first.

Single-Transaction Test

```
use my_pkg::hero;
use sui::test_scenario as ts;
use sui::test_utils;

#[test]
fun test_mint_single_tx() {
    let admin = @0xA;
    let mut scenario = ts::begin(admin);

    let hero = hero::mint_for_tests(scenario.ctx());
    assert!(hero.level == 1);
    test_utils::destroy(hero);

    scenario.end();
}
```

When to use:

- simple positive-path check in one tx context (no `next_tx(...)` needed).

Multi-Transaction Scenario Test

```
use my_pkg::hero;
use sui::test_scenario as ts;

#[test]
fun test_two_users_flow() {
    let admin = @0xA;
    let user = @0xB;
    let mut scenario = ts::begin(admin);

    hero::mint_to(user, scenario.ctx());

    scenario.next_tx(user);
    let mut h = scenario.take_from_sender<hero::Hero>();
    hero::level_up(&mut h);
    ts::return_to_sender(h);

    scenario.end();
}
```

When to use:

- tests that need `next_tx(...)`, sender changes, or step-by-step flow checks.

Reusing Module Error Constants

```
use my_pkg::hero::{Self, ENotAdmin};
use sui::test_scenario as ts;

#[test]
#[expected_failure(abort_code = ENotAdmin)]
fun test_non_admin_fails() {
    let admin = @0xA;
    let attacker = @0xB;
    let mut scenario = ts::begin(admin);

    hero::init_admin_only(scenario.ctx());

    scenario.next_tx(attacker);
    hero::admin_only_call(scenario.ctx()); // expected abort

    scenario.end();
}
```

When to use:

- validating exact failure reason and guarding regressions.

Annotation Cheatsheet (Use / Avoid)

```
#[test_only]              // for helper modules/functions only in tests
#[test]                   // marks a unit test function
#[expected_failure(...)] // test must fail with expected abort
```

Use:

- `#[test_only]` for test scaffolding not meant for production publish.
- `#[test]` for executable test cases.
- `#[expected_failure]` for negative-path assertions.

Avoid:

- putting production logic under `#[test_only]`.
- overusing `#[expected_failure]` for happy paths.
- duplicating module constants in tests; import and reuse them.

Command loop: `sui move test` → fix → rerun.

Deployment & CLI Mastery

Module 5

CLI Essentials

From local setup to on-chain interaction.

```
sui client new-address ed25519                  # Create a new wallet address
sui client addresses                            # List local wallet addresses
sui client active-address                       # Show current active wallet
sui client switch --address <ADDRESS>           # Switch active wallet address
sui client envs                                 # Show configured networks
sui client active-env                           # Show active network
sui client switch --env devnet                  # Switch network to devnet
sui client switch --env testnet                 # Switch network to testnet
sui client faucet                               # Request faucet gas (devnet/testnet)
sui client faucet --address <ACTIVE_ADDRESS>    # Request faucet for specific address
sui client gas                                  # Show gas objects / balances
```

Network Switching + Publish + Call

```
sui client envs                                  # Show configured environments
sui client active-env                            # Confirm active network
sui client switch --env devnet                   # Switch between networks
sui client switch --env testnet                  # Switch between networks
```

Then deploy and interact:

```
sui client publish --gas-budget 100000000        # Publish the package
sui client call \                                # Call a function on-chain
  --package <PACKAGE_ID> \
  --module hero \
  --function mint \
  --gas-budget 10000000
```

Then verify the object ID on suiexplorer.com.

Verify On-Chain Objects

Use any explorer to confirm your package, transaction digest, and created object IDs:

- SuiVision
- SuiScan

Quick URL patterns (replace placeholders):

```
https://suivision.xyz/object/<OBJECT_ID>?network=testnet
https://suiscan.xyz/testnet/object/<OBJECT_ID>
```

Tip: always match explorer network (testnet/devnet/mainnet) with your `sui client active-env`.

Hackathon Reveal

Turning Community Contributions into Proof of Work

Hackathon Brief

The Challenge

Turning Community Contributions into Proof of Work.

Systemic Problem

Work done in communities is real work, but recognition systems are built for formal jobs, not informal contribution.

As a result, high-effort contributors remain "invisible" in hiring pipelines.

Why This Is a Real Problem

Without verifiable records, opportunity goes to people with strong networks, not always strong contribution. This creates a trust gap between talent and employers.

Background for the Challenge

Across schools, DAOs, guilds, and learning communities, contribution data is fragmented and rarely portable.

Your mission: design a system where contribution claims become trusted, portable proof people can use for jobs and gigs.

Homework

Head to `https://github.com/MystenLabs/ygg-sui` clone it and look for slides/homework.pdf
This repo is fitted with rules and guardrails, conduct your development and prompting inside this repo to build your homework.

Day 2

The Builder

Bridging Move to the World

Sui TypeScript SDK

Module 1

Module 1 Scope

What This Module Covers

Core SDK setup with the Sui gRPC client:

- Client initialization
- Network gRPC URLs
- Funding from faucet

What Is the Sui gRPC Client?

What

The TypeScript app (frontend or backend service) uses `SuiGrpcClient` to send requests to a Sui fullnode.
A fullnode is a Sui node that stores blockchain state and exposes APIs to read data and submit signed transactions.

Benefits

Benefits:

- typed request/response objects in TypeScript
- one client handles both reads (`client.core.getBalance`, `client.core.getObject`) and writes (`client.core.executeTransaction`)
- easy integration with wallet sign flow for user-authorized writes

*Do not query fullnodes for every dashboard render at scale; use an indexer or backend cache for analytics-heavy views.

SuiGrpcClient Initialization

```
import { SuiGrpcClient } from "@mysten/sui/grpc";

const client = new SuiGrpcClient({
  network: "testnet",
  baseUrl: "https://fullnode.testnet.sui.io:443",
});
```

Build notes:

1. Create one `SuiGrpcClient` instance and reuse it; avoid creating a new client per request.
2. Bind network URL from environment variables, not inline constants in feature code.
3. You can print active network + URL at startup for verbosity.

gRPC URLs and Network Selection

Typical URLs

- Devnet: `https://fullnode.devnet.sui.io:443`
- Testnet: `https://fullnode.testnet.sui.io:443`
- Mainnet: `https://fullnode.mainnet.sui.io:443`
- Localnet: your local fullnode endpoint

What To Note While Building

- wallet network, client URL, and faucet network must be the same (for example: all `testnet`)
- public endpoints can rate-limit; implement retries/backoff for read calls
- store endpoint URLs in environment config (or a single source) to prevent accidental mismatched usage (etc testnet for network, devnet for client url)

SuiGrpcClient Initialization (Improved)

```
const network = process.env.NETWORK;

const GRPC_URLS = {
  mainnet: "https://fullnode.mainnet.sui.io:443",
  testnet: "https://fullnode.testnet.sui.io:443",
  devnet: "https://fullnode.devnet.sui.io:443",
};

export const client = new SuiGrpcClient({
  network,
  baseUrl: GRPC_URLS[network],
});
```

Faucet Usage (Dev/Test Only)

Why Use Faucet

Faucet sends free testnet/devnet SUI gas coins to a wallet address.
Use faucet when students need gas to execute on-chain write transactions during exercises.

When Not To Use

Do not design production funding flows around faucet calls; faucet exists only for test networks.
Take note:

- faucet can rate limit
- requests can be delayed

Quick CLI check:

```
sui client active-env            # should be devnet or testnet
sui client active-address        # wallet receiving faucet funds
sui client faucet                # request gas
sui client gas                   # confirm gas objects/balance
```

Sui TS gRPC Read + Write Queries

Module 2

Module 2

Read Track

How to query objects and inspect returned data with the right options.

Write Track

How to build write transactions with PTBs, and when PTB is the correct transaction shape.

How to Read from Chain

Read Query Mental Model

1. Pick resource type: object, coin balance, transaction, or event.
2. Pick identifier/filter: owner address, object ID, package, module, or struct type.
3. Request only fields the current screen/action needs.

Build Notes

- minimize payload size to reduce latency
- paginate for lists (`cursor`, `limit`)
- after a write, account for short propagation delay before follow-up reads
- map node/RPC errors to actionable UI messages (retry, switch network, invalid input)

Reads That Matter (for now)

```ts
const { response } = await client.stateService.listOwnedObjects({
  owner: address,
  // depends on SDK version/protobuf options:
  // include filters and content flags as needed
});

const balanceRes = await client.core.getBalance({
  owner: address,
  coinType: "0x2::sui::SUI", // optional; defaults to SUI
});

const objectRes = await client.core.getObject({
  objectId: "<OBJECT_ID>",
  options: {
    showType: true,
    showOwner: true,
    showContent: true,
  },
});
```

Reading Objects End-to-End

1. Read object IDs with `stateService.listOwnedObjects`.
2. For a selected object ID, call `client.core.getObject`.
3. Validate ownership/type before allowing user action.
4. Build UI state from returned on-chain data, not stale local assumptions.

Why this matters:

- prevents stale/off-chain mistakes
- blocks invalid actions before wallet signing
- ensures PTB inputs use correct object IDs and expected types

Writing with PTBs

What Is a PTB?

A **Programmable Transaction Block (PTB)** is one transaction containing an ordered list of commands.
Validators execute commands in order; if any command fails, the entire transaction is reverted.
PTB behaves as an **atomic transaction**:

- **all-or-nothing** commit for the whole command sequence
- no partial on-chain state from half-finished business logic
- either final effects/events are all recorded, or none are

Why Use PTB

Benefits:

- run multiple dependent operations in one signed transaction
- reduce partial-state bugs (all succeed or all fail)
- one wallet approval can complete one user intent
- treat a multi-step user action as one atomic unit (approve once, commit once)

PTB Example

```
import { Transaction } from "@mysten/sui/transactions";

const tx = new Transaction();
tx.moveCall({
  target: "0xabc::proof_of_work::submit_claim",
  arguments: [tx.object(programId), tx.pure.address(contributor)],
});
```

Build notes:

1. Pre-read and validate every object input before constructing PTB.
2. Keep PTBs focused; avoid bundling unrelated business actions.
3. Surface dry-run/simulation errors clearly to the user.
4. Confirm effects from `objectChanges` / `events` in execution results, not optimistic UI alone.

PTB Argument Encoding

Use the right argument builder based on the Move function parameter type:

- `tx.object(...)` for on-chain object inputs (`&T`, `&mut T`, `T has key`)
- `tx.pure.*(...)` for non-object values (serialized primitive data)

```
import { Transaction } from "@mysten/sui/transactions";

const tx = new Transaction();

tx.moveCall({
  target: "0xabc::proof_of_work::submit_claim_with_metadata",
  arguments: [
    tx.object(programId), // object input
    tx.pure.address(contributorAddress), // address
    tx.pure.u64(hoursContributed), // u64
    tx.pure.bool(mentorVerified), // bool
    tx.pure.string(contributionTitle), // string
  ],
});
```

PTB Example: Chaining Returned Objects

```
// Command 1: create a claim in Pending state
const pendingClaim = tx.moveCall({
  target: "0xabc::proof_of_work::submit_claim",
  arguments: [tx.object(programId), tx.pure.address(contributor)],
});

// Command 2: transition claim -> Verified
const verifiedClaim = tx.moveCall({
  target: "0xabc::proof_of_work::attest_claim",
  arguments: [pendingClaim, tx.object(verifierCapId)],
});

// Command 3: mint portable credential from verified claim
const credential = tx.moveCall({
  target: "0xabc::proof_of_work::mint_credential",
  arguments: [
    tx.object(programId),
    verifiedClaim,
    tx.pure.address(contributor),
  ],
});
```

PTB Signing Flow

Use the explicit 3-step flow when you want clearer control over signing/execution boundaries.

```
import { Transaction } from "@mysten/sui/transactions";
import { SuiGrpcClient } from "@mysten/sui/grpc";

const suiGrpcClient = new SuiGrpcClient({ network: "testnet" });

const tx = new Transaction();
...

// 1) Build transaction bytes
const txBytes = await tx.build({ client: suiGrpcClient });

// 2) Sign with a signer (keypair or wallet signer)
const { signature } = await signer.signTransaction(txBytes);

// 3) Execute using bytes + signature
const result = await suiGrpcClient.core.executeTransaction({
  // exact payload fields vary by SDK patch version; keep this shape as teaching pseudocode
  transaction: txBytes,
  signatures: [signature],
});
```

Live Coding: Bootstrap a Sui dApp Repo (Vite + React)

Module 3

Module 3 Try-It-Yourself Commands

Scaffold a new app from the MystenLabs Sui dApp Vite.js template:

```
bun create @mysten/dapp@latest
cd counter-dapp
bun install
bun run dev
```